

Lab Assignment Number: 19.5

Name: Hemavathi N

HT No:2303A51965

Batch No:24

Task 1 – Python List Processing → C++ STL Conversion

Task:

Translate a Python program that filters even numbers from a list into C++.

Instructions:

- Provide AI with Python list comprehension code.
- Convert it into C++ using vectors and loops/STL.
- Verify outputs for the same input list.

Expected Output:

- Equivalent working C++ program using STL.

```
1  # generate a python code that filters even numbers from a list
2  from ast import main
3
4
5  def filter_even_numbers(numbers):
6      """Filter even numbers from a list.
7
8      Args:
9          numbers (list): A list of integers.
10     Returns:
11         list: A list of even integers.
12     """
13     even_numbers = [num for num in numbers if num % 2 == 0]
14     return even_numbers
15
16 # Example usage
17 if __name__ == "__main__":
18     sample_numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
19     even_numbers = filter_even_numbers(sample_numbers)
20     print("Even numbers:", even_numbers)
21
22 # convert the above code to c++ using vectors and loops/STL
23 #include <iostream>
24 #include <vector>
25 #include <algorithm>
26 std::vector<int> filterEvenNumbers(const std::vector<int>& numbers) {
27     std::vector<int> evenNumbers;
28     for (int num : numbers) {
29         if (num % 2 == 0) {
30             evenNumbers.push_back(num);
31         }
32     }
33     return evenNumbers;
34 }
35
36 int main() {
37     std::vector<int> sampleNumbers = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
38     std::vector<int> evenNumbers = filterEvenNumbers(sampleNumbers);
39     std::cout << "Even numbers: ";
40     for (int num : evenNumbers) {
41         std::cout << num << " ";
42     }
43     std::cout << std::endl;
44     return 0;
45 }
```

Task 2 – Java to Python OOP Translation

Task:

Convert a Java Student class into an equivalent Python class.

Instructions:

- Translate attributes, constructor, and methods properly.
- Ensure Python version uses `__init__` and follows Python conventions.
- Test both versions with sample objects.

Expected Output:

- Correct Python OOP class translated from Java

```
1  # generate a java code with student class and respective attributes
2  public class Student {
3      private String name;
4      private int age;
5      private String studentId;
6
7      public Student(String name, int age, String studentId) {
8          this.name = name;
9          this.age = age;
10         this.studentId = studentId;
11     }
12
13     public String getName() {
14         return name;
15     }
16
17     public void setName(String name) {
18         this.name = name;
19     }
20
21     public int getAge() {
22         return age;
23     }
24
25     public void setAge(int age) {
26         this.age = age;
27     }
28
29     public String getStudentId() {
30         return studentId;
31     }
32
33     public void setStudentId(String studentId) {
34         this.studentId = studentId;
35     }
36 }
```

```

37 #convert the above java code to python code
38 class Student:
39     def __init__(self, name: str, age: int, student_id: str):
40         self.name = name
41         self.age = age
42         self.student_id = student_id
43
44     def get_name(self) -> str:
45         return self.name
46
47     def set_name(self, name: str):
48         self.name = name
49
50     def get_age(self) -> int:
51         return self.age
52
53     def set_age(self, age: int):
54         self.age = age
55
56     def get_student_id(self) -> str:
57         return self.student_id
58
59     def set_student_id(self, student_id: str):
60         self.student_id = student_id
61
62 # Example usage
63 student = Student("Alice", 20, "S12345")
64 print(student.get_name()) # Output: Alice
65 print(student.get_age()) # Output: 20
66 print(student.get_student_id()) # Output: S12345
67 student.set_name("Bob")
68 print(student.get_name()) # Output: Bob
69 student.set_age(21)
70 print(student.get_age()) # Output: 21
71 student.set_student_id("S54321")
72 print(student.get_student_id()) # Output: S54321

```

Task 3 – SQL Aggregation Query → Pandas GroupBy

Task:

Convert an SQL query with aggregation into Pandas.

Example SQL:

SELECT department, AVG(salary)

FROM employees

GROUP BY department;

Instructions:

- Translate into Pandas groupby() and mean().
- Validate using sample DataFrame.

Expected Output:

- Equivalent Pandas aggregation code.

```

1  -- Create a SQL query to find the average salary of employees in each department from a table named 'employees'. Group the results by department
2  SELECT department, AVG(salary)
3  FROM employees
4  GROUP BY department;
5  #convert the above SQL query to a Python code using pandas library
6  import pandas as pd
7  # Assuming you have a DataFrame named 'employees' with columns 'department' and 'salary'
8  average_salary_by_department = employees.groupby('department')['salary'].mean().reset_index()
9  print(average_salary_by_department)
10 |
11

```

Task 4 – Python File Handling → C++ File Streams

Task:

Convert a Python program that reads a text file and counts words into C++.

Instructions:

- Translate open() usage into ifstream.
- Handle file-not-found errors properly.
- Compare outputs for the same file.

Expected Output:

- Working C++ word counter program.

```

1  # generate a Python program that reads a text file and counts words
2  def count_words_in_file(file_path):
3      try:
4          with open(file_path, 'r') as file:
5              text = file.read()
6              words = text.split()
7              word_count = len(words)
8              print(f'The number of words in the file is: {word_count}')
9      except FileNotFoundError:
10         print(f'The file {file_path} was not found.')
11     except Exception as e:
12         print(f'An error occurred: {e}')
13 if __name__ == "__main__":
14     file_path = input("Enter the path to the text file: ")
15     count_words_in_file(file_path)
16 # Convert above Python program that reads a text file and counts words to c++ program
17 #include <iostream>
18 #include <fstream>
19 #include <sstream>
20 #include <string>
21 using namespace std;
22 void countWordsInFile(const string& filePath) {
23     ifstream file(filePath);
24     if (!file.is_open()) {
25         cout << "The file " << filePath << " was not found." << endl;
26         return;
27     }
28     string word;
29     int wordCount = 0;
30     while (file >> word) {
31         wordCount++;
32     }
33     cout << "The number of words in the file is: " << wordCount << endl;
34     file.close();
35 }
36 int main() {
37     string filePath;
38     cout << "Enter the path to the text file: ";
39     getline(cin, filePath);
40     countWordsInFile(filePath);
41     return 0;
42 }

```

Task 5 – Java Exception Handling → Python Try/Except

Task:

Translate a Java program using try-catch for division into Python.

Instructions:

- Convert exception syntax properly.
- Handle divide-by-zero gracefully.
- Verify both programs behave the same.

Expected Output:

- Equivalent Python error-handling program.

```
1  # generate a Java program using try-catch for division
2  public class Division {
3      public static void main(String[] args) {
4          int numerator = 10;
5          int denominator = 0;
6
7          try {
8              int result = numerator / denominator;
9              System.out.println("Result: " + result);
10         } catch (ArithmeticException e) {
11             System.out.println("Error: Division by zero is not allowed.");
12         }
13     }
14 }
15 # convert the above Java program to Python using try-except for division
16 def main():
17     numerator = 10
18     denominator = 0
19
20     try:
21         result = numerator / denominator
22         print("Result:", result)
23     except ZeroDivisionError:
24         print("Error: Division by zero is not allowed.")
25 if __name__ == "__main__":
26     main()
```